

1. PROJECT OVERVIEW

- **Project Title:** FitBuddy - AI Workout Generator

2. INTRODUCTION :

This document outlines the primary Python implementation utilized to orchestrate the FitBuddy platform. As designed, the core solution involves a Python backend processing HTML form data and passing it to the Gemini API (`gemini-3.5-flash-lite`). Exception handling and a strict 10-second timeout parameter are implemented to ensure the server doesn't hang during API latency, providing a seamless user experience.

3. CORE IMPLEMENTATION LOGIC

A. Database Schema Initialization (`database.py`)

The SQLAlchemy Object-Relational Mapper (ORM) is utilized to define the data structures for user profiles and their corresponding AI-generated plans.

Python

```
class User(Base):
    __tablename__ = "users"
    id = Column(Integer, primary_key=True, index=True)
    name = Column(String)
    age = Column(Integer)
    weight = Column(Float)
    goal = Column(String)
    intensity = Column(String)

class WorkoutPlan(Base):
    __tablename__ = "workout_plans"
    id = Column(Integer, primary_key=True, index=True)
    user_id = Column(Integer)
    original_plan = Column(String)
```

B. Generative AI Orchestration (`gemini_generator.py`)

This script manages the secure initialization of the Google Gemini API. It includes the newly updated exception handling logic and a 10-second timeout to manage network latency gracefully without crashing the main application thread.

Python

```
import google.generativeai as genai
import os

genai.configure(api_key=os.environ.get("GOOGLE_API_KEY"))
model = genai.GenerativeModel("gemini-3.5-flash-lite")

def generate_workout_gemini(user_input):
    prompt = f"Create a structured 7-day workout plan for {user_input['goal']} at {user_input['intensity']} intensity."
    try:
        # 10-second timeout implemented to prevent server hanging
        response = model.generate_content(prompt, request_options={"timeout": 10})
        return response.text
    except Exception as e:
        return f"Error: {e}"
```

C. Application Routing Controller (`routes.py`)

The FastAPI route manages the POST request, processes the user inputs, and merges the data with the Jinja2 frontend template.

Python

```
@router.post("/generate-workout",
response_class=HTMLResponse)
def generate_workout(request: Request, user_id:
```

```
int=Form(...), goal: str=Form(...), intensity:
str=Form(...)):
    # Database Logic
    db = SessionLocal()

    # AI Generation
    user_input = {"goal": goal, "intensity": intensity}
    plan = generate_workout_gemini(user_input)

    # Save & Render
    return templates.TemplateResponse("result.html",
{"request": request, "workout_plan": plan})
```